Dedicated KVM Multi-Node Kubernetes Deployment of the Solana Kubernetes Observability Core

Oleksandr Khoshaba
ORCID: 0000-0001-5375-6280

Version 0.4.0
2026
Technical report DOI: 10.5281/zenodo.20561100

Abstract

This technical report documents the deployment and validation of the Solana Kubernetes Observability Core on a dedicated KVM-based multi-node Kubernetes cluster. The work extends the previous Minikube-based Kubernetes validation by deploying the same observability-oriented Solana workload on a kubeadm cluster composed of one control-plane virtual machine and two worker virtual machines. The validated deployment includes a Solana validator StatefulSet, validator ledger PersistentVolumeClaim, wallet-init Job, latency monitor Deployment, RPC Service, Yellowstone/Geyser gRPC Service, metrics Service, and wallet transfer validation Job. The stage remains focused on infrastructure and observability.

Contents

1 Introduction

The Solana Containerised Testbed is a reproducible local Solana environment for infrastructure, observability, dataset generation, and latency/performance research on legacy x86_64 hardware. Earlier stages established the containerised no-AVX2 Solana local testbed [1], added the Yellowstone/Geyser observability extension [5], and validated the Kubernetes Observability Core in a Minikube environment [3, 2].

This report documents the next stage: the deployment of the Solana Kubernetes Observability Core on a dedicated KVM-based multi-node Kubernetes cluster. The implementation and evidence for this stage are released as Solana Containerised Testbed v0.4.0 [4].

2 Project Evolution

The project evolved through several stages:

1. Compose-based local Solana testbed.
2. No-AVX2 compatibility workflow.
3. Yellowstone/Geyser observability extension.
4. Minikube-based Kubernetes validation.
5. Dedicated KVM multi-node Kubernetes deployment.

3 Host and Virtualisation Environment

The deployment target is a CentOS Stream 9 host with KVM/libvirt support. VM disks are placed under the host home filesystem rather than the smaller root filesystem.

4 Kubernetes Cluster Topology

The validated cluster uses one control-plane VM and two worker VMs.

Table 1: Dedicated KVM Kubernetes cluster topology.

Node	Role	Label	Main purpose
`k8s-cp-01`	control-plane	`testbed-role=control-plane`	Kubernetes API and control-plane services.
`k8s-worker-01`	worker	`testbed-role=validator`	Solana validator workload and validator ledger PVC.
`k8s-worker-02`	worker	`testbed-role=observability`	Monitor workload and auxiliary validation jobs.

5 Kubernetes Bootstrap and Ansible Automation

The infrastructure automation is stored under:

```
infra/ansible/kvm-kubeadm/
```

The automation covers node preparation, containerd installation, Kubernetes tool installation, control-plane initialisation, worker join, CNI installation, node labelling, local-path storage installation, validation, and reset support.

5.1 Main Ansible Entry Points

The node preparation entry point combines the bootstrap, operating-system preparation, container runtime installation, Kubernetes tooling installation, and readiness validation steps.

```
---
- import_playbook: 00-bootstrap-python.yml
- import_playbook: 10-prepare-os-for-k8s.yml
- import_playbook: 20-install-containerd.yml
- import_playbook: 30-install-kubernetes-tools.yml
- import_playbook: 90-validate-kubeadm-readiness.yml
```

The cluster creation entry point initialises the control plane, installs Calico, joins the workers, applies node labels, and installs the local-path provisioner.

```
---
- import_playbook: 40-init-control-plane.yml
- import_playbook: 50-install-calico-direct.yml
- import_playbook: 60-join-workers.yml
- import_playbook: 70-label-k8s-nodes.yml
- import_playbook: 80-install-local-path-provisioner.yml
```

The reset playbook documents the teardown path. This is important because a reproducible infrastructure baseline must support both deployment and reset.

```
---
- name: Reset kubeadm cluster on all nodes
  hosts: k8s
  become: true
  gather_facts: false

  tasks:
    - name: Run kubeadm reset
      ansible.builtin.command: kubeadm reset -f
      changed_when: true
      failed_when: false

    - name: Stop kubelet
      ansible.builtin.systemd:
        name: kubelet
        state: stopped
      failed_when: false

    - name: Stop containerd
      ansible.builtin.systemd:
        name: containerd
        state: stopped
      failed_when: false

    - name: Remove Kubernetes directories
```

```
    ansible.builtin.file:
      path: "{{ item }}"
      state: absent
    loop:
      - /etc/kubernetes
      - /var/lib/kubelet
      - /var/lib/etcd
      - /var/lib/cni
      - /etc/cni
      - /opt/cni
      - /root/.kube
      - /home/khoshaba/.kube

  - name: Remove Calico state
    ansible.builtin.file:
      path: "{{ item }}"
      state: absent
    loop:
      - /var/lib/calico
      - /var/run/calico

  - name: Remove CNI network interfaces if present
    ansible.builtin.shell: |
      set +e
      ip link delete cni0 2>/dev/null || true
      ip link delete flannel.1 2>/dev/null || true
      ip link delete tunl0 2>/dev/null || true
      ip link delete vxlan.calico 2>/dev/null || true
    args:
      executable: /bin/bash
    changed_when: false

  - name: Clean iptables Kubernetes and CNI rules
    ansible.builtin.shell: |
      set +e
      iptables -F
      iptables -t nat -F
      iptables -t mangle -F
      iptables -X
    args:
      executable: /bin/bash
    changed_when: true

  - name: Restart containerd
    ansible.builtin.systemd:
      name: containerd
      state: started
      enabled: true

  - name: Enable kubelet but keep it stopped before kubeadm init or join
    ansible.builtin.systemd:
      name: kubelet
      enabled: true
      state: stopped
    failed_when: false
```

6 Solana Kubernetes Observability Core

The portable Kubernetes manifests are stored under:

```
k8s/base/
```

The deployed core includes the validator StatefulSet, validator ledger PVC, wallet-init Job, monitor Deployment, RPC Service, Yellowstone/Geyser gRPC Service, and metrics Service.

7 Kustomize Overlay for KVM Multi-Node Deployment

The dedicated KVM multi-node overlay is stored under:

```
k8s/overlays/kvm-multinode/
```

The overlay adds validator placement on `testbed-role=validator`, monitor placement on `testbed-role=observability`, and explicit `local-path` storage for the validator ledger PVC.

8 Kustomize Overlay Listings

The dedicated KVM multi-node deployment is represented by the `k8s/overlays/kvm-multinode` overlay. The overlay keeps the base manifests portable and adds only the environment-specific scheduling and storage rules.

8.1 Overlay Kustomization

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - ../../base

patches:
  - path: patch-validator-placement.yaml
    target:
      kind: StatefulSet
      name: solana-validator

  - path: patch-monitor-placement.yaml
    target:
      kind: Deployment
      name: solana-monitor

  - path: patch-validator-ledger-pvc.yaml
    target:
      kind: PersistentVolumeClaim
      name: validator-ledger-pvc
```

8.2 Validator Placement Patch

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: solana-validator
spec:
```

```
  template:
    spec:
      nodeSelector:
        testbed-role: validator
```

8.3 Monitor Placement Patch

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: solana-monitor
spec:
  template:
    spec:
      nodeSelector:
        testbed-role: observability
```

8.4 Validator Ledger PVC Patch

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: validator-ledger-pvc
spec:
  storageClassName: local-path
```

9 Validation Evidence

The validation evidence is stored in the repository under:

```
results/kvm-kubeadm-cluster/
results/kvm-multinode-observability-core/
```

Selected evidence files were also copied into the report directory to make the technical report reproducible from the LaTeX source tree.

9.1 Kubernetes Node Readiness and Placement Labels

The cluster contains one control-plane node and two worker nodes. The worker nodes are labelled according to their intended testbed role.

```
\lstinputlisting{evidence/nodes-labels.txt}
```

The validator workload is assigned to the node labelled `testbed-role=validator`. The observability workload is assigned to the node labelled `testbed-role=observability`. This explicit placement is the main difference from the earlier Minikube validation, where all workloads were scheduled onto a single-node cluster.

9.2 StorageClass Validation

The validator ledger PVC uses the `local-path` StorageClass. This is appropriate for the first dedicated KVM baseline because the validator ledger is intentionally node-local and bound to the validator worker.

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
local-path (default) rancher.io/local-path Delete WaitForFirstConsumer false 7h52m
```

9.3 Workload Placement

The deployed pods confirm the intended placement: the validator pod runs on `k8s-worker-01`, while the monitor runs on `k8s-worker-02`.

```
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
solana-monitor-f995c8b9d-95vmp 1/1 Running 0 10m 192.168.118.65 k8s-worker-02 <none> <
    none>
solana-validator-0 1/1 Running 0 10m 192.168.36.198 k8s-worker-01 <none> <none>
wallet-init-ngm2v 0/1 Completed 0 10m 192.168.118.66 k8s-worker-02 <none> <none>
```

9.4 Services and EndpointSlices

The deployment exposes three internal services: RPC on port `8899`, Yellowstone/Geyser gRPC on port `10000`, and metrics on port `9464`.

```
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE SELECTOR
solana-geyser-grpc ClusterIP 10.96.199.115 <none> 10000/TCP 10m app.kubernetes.io/
    component=validator,app.kubernetes.io/name=solana-containerised-testbed
solana-metrics ClusterIP 10.110.176.165 <none> 9464/TCP 10m app.kubernetes.io/
    component=monitor,app.kubernetes.io/name=solana-containerised-testbed
solana-rpc ClusterIP 10.96.72.163 <none> 8899/TCP 10m app.kubernetes.io/component=
    validator,app.kubernetes.io/name=solana-containerised-testbed
solana-validator-headless ClusterIP None <none> 8899/TCP,10000/TCP 10m app.kubernetes.
    io/component=validator,app.kubernetes.io/name=solana-containerised-testbed
```

EndpointSlice evidence confirms that the services resolve to the expected pod endpoints.

```
NAME ADDRESSTYPE PORTS ENDPOINTS AGE
solana-geyser-grpc-glkn8 IPv4 10000 192.168.36.198 10m
solana-metrics-p8647 IPv4 9464 192.168.118.65 10m
solana-rpc-rp9kf IPv4 8899 192.168.36.198 10m
solana-validator-headless-xj4rb IPv4 10000,8899 192.168.36.198 10m
```

9.5 RPC Health

The RPC health check returned `ok` through the Kubernetes Service endpoint.

```
{"jsonrpc":"2.0","result":"ok","id":1}
```

9.6 Wallet Transfer Validation

The wallet transfer validation was executed as a Kubernetes Job. It created a temporary sender wallet, requested an airdrop, transferred `0.5 SOL` to the receiver, confirmed the transaction, and printed the final balances.

The validation Job name was recorded as follows:

```
wallet-transfer-validation-kvm-20260604035307
```

The completed Kubernetes Job was recorded as follows:

```
NAME STATUS COMPLETIONS DURATION AGE CONTAINERS IMAGES SELECTOR
wallet-transfer-validation-kvm-20260604035307 Complete 1/1 17s 26s wallet-transfer-
    validation docker.io/khoshaba/solana-localnet-validator:v1.18.25-noavx2-ivybridge-
    yellowstone batch.kubernetes.io/controller-uid=bf232e37-71e1-4ef5-9f3b-
    ba9a0569789e
```

The completed validation pod was recorded as follows:

```
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
wallet-transfer-validation-kvm-20260604035307-vcv2b 0/1 Completed 0 40s 192.168.118.73
     k8s-worker-02 <none> <none>
```

The validation confirmed that the sender balance changed from `2 SOL` to `1.499995 SOL`, while the receiver balance changed from `10000 SOL` to `10000.5 SOL`.

9.7 Metrics After Wallet Transfer

After the transfer validation, the metrics endpoint reported both slot interval observations and a transaction latency sample.

```
# HELP solana_slot_interval_seconds Time interval between consecutive Solana slots,
    used to measure slot production stability.
# TYPE solana_slot_interval_seconds summary
solana_slot_interval_seconds{quantile="0.5"} 0.411585126
solana_slot_interval_seconds{quantile="0.9"} 0.415077648
solana_slot_interval_seconds{quantile="0.99"} 0.41908748
solana_slot_interval_seconds_sum 1811.0787084130004
solana_slot_interval_seconds_count 4397
# HELP solana_transaction_latency_seconds Latency distribution from transaction
    broadcast to confirmation.
# TYPE solana_transaction_latency_seconds summary
solana_transaction_latency_seconds{quantile="0.5"} 0.318976568
solana_transaction_latency_seconds{quantile="0.95"} 0.318976568
solana_transaction_latency_seconds{quantile="0.99"} 0.318976568
solana_transaction_latency_seconds_sum 0.318976568
solana_transaction_latency_seconds_count 1
```

The important line is:

```
solana_transaction_latency_seconds_count 1
```

This confirms that the monitor observed at least one transaction latency sample in the dedicated KVM multi-node deployment.

10 Scope and Limitations

This report documents infrastructure and observability validation only. Load generation, dashboarding, Prometheus/Grafana integration, MPC, reinforcement learning, MARL, and Agave are out of scope.

11 Conclusion

The Solana Kubernetes Observability Core has been successfully deployed and validated on a dedicated KVM-based multi-node Kubernetes cluster.

References

[1] Oleksandr Khoshaba. Containerising a Solana v1.18.25 Local Testbed for Reproducible Dataset Generation on Legacy x86_64 CPUs, 2026.

[2] Oleksandr Khoshaba. From Compose to Kubernetes: Validating the Observability Core of a No-AVX2 Solana Containerised Testbed, 2026.

[3] Oleksandr Khoshaba. Solana Containerised Testbed v0.3.0: Kubernetes Observability Core, 2026.

[4] Oleksandr Khoshaba. Solana Containerised Testbed v0.4.0: Dedicated KVM Multi-Node Kubernetes Deployment, 2026.

[5] Oleksandr Khoshaba. Yellowstone/Geyser Observability Extension for the Solana Containerised Testbed v0.2.0, 2026.